

AKB, Ilmwiki, GBrain 비교 및 AKB 발전 방향

- 작성일: 2026-04-18
- 목적: agent memory를 knowledge base화하는 세 가지 접근, 즉 AKB, Ilmwiki, GBrain을 비교하고, 그 장단점을 반영한 AKB 발전 방향을 제안한다.
- 전제: AKB는 공개 정보가 상대적으로 적어, 아래 평가는 공개 확인 가능한 정보 기준으로 작성했다.

1. Executive Summary

핵심 결론은 아래와 같다.

1. 장기기억 자체를 가장 강하게 구현한 것은 GBrain이다.
2. 사람과 AI가 함께 읽고 공유하는 knowledge base로는 Ilmwiki가 가장 적합하다.
3. 여러 agent가 공용 memory backend를 공유하는 구조에는 AKB가 가장 자연스럽다.
4. 따라서 AKB의 가장 유망한 발전 방향은 단순 저장소를 넘어서, 공유 memory layer + 자동 정리/유지보수 + publishable wiki layer를 결합하는 것이다.

2. 한눈에 보는 점수 비교

점수는 5점 만점이다. 5점이 가장 강하다.

평가 항목	AKB	Ilmwiki	GBrain
장기기억 운영력	3.0	3.5	5.0
사람 공유용 KB 적합성	2.5	5.0	3.5
multi-agent shared backend 적합성	5.0	2.5	4.0
자동 maintenance / enrichment	2.5	3.5	5.0
도입 및 운영 단순성	4.0	4.0	2.5
결과물의 가독성 / 배포성	2.5	5.0	3.0

3. 전략 관점 비교표

항목	AKB	Ilmwiki	GBrain
핵심 역할	MCP 기반 shared memory backend	세션/노트를 interlinked wiki로 컴파일	agent brain 운영체계에 가까운 memory OS
주 저장 단위	DB 기반 memory entry, collection, shared context	markdown page, session transcript, static site	brain page, entity, report, task, recurring job
최적 사용자	여러 agent / app이 같은 memory를 공유하는 팀	사람과 AI가 함께 읽는 문서형 knowledge base가 필요한 팀	proactive agent를 장기 운영하려는 개인/팀
장기 기억 방식	저장 및 검색 중심	정리된 지식 결과물 축적 중심	ingest, enrich, maintain, query의 순환 구조
사람 친화적 가독성	제한적	매우 강함	중간
자동 유지보수	상대적으로 약함	lint, confidence, lifecycle 중심	매우 강함, cron 및 유지보수 루프 내장
공유 방식	MCP 중심 공유	정적 사이트, Ilms.txt, JSON-LD 등 문서 배포에 강함	MCP, publish 기능, brain-first agent 운영
리스크	제품 계층이 얇으면 단순 벡터 스토어처럼 보일 수 있음	runtime memory보다는 compiled artifact에 가까움	구조가 무겁고 운영 복잡도가 높음

4. 세부 비교

4.1 장기 기억 관점

AKB

- 강점
 - 여러 agent가 같은 memory를 조회하고 갱신하는 *공용 기억 저장소* 역할에 적합하다.
 - PostgreSQL + pgvector 기반으로 보이며, 실용적이고 확장 가능한 backend 설계와 잘 맞는다.
 - MCP 인터페이스 중심이라 agent ecosystem과 붙이기 쉽다.
- 약점
 - 공개 정보 기준으로는 *enrichment, citation repair, staleness 관리, entity lifecycle* 같은 고차 memory 운영 기능이 약하게 보인다.
 - 따라서 현재 상태만으로는 "잘 만든 shared memory DB"에 가깝고, "스스로 기억을 가꾸는 brain"과는 거리가 있다.

Ilmwiki

- 강점
 - 단순 저장이 아니라 *knowledge를 재구성한 위키*를 남긴다.
 - 세션 기록이 사라지지 않고, 문서화된 지식으로 누적된다.
 - confidence, lifecycle, lint가 있어 지식 품질 관리가 가능하다.
- 약점
 - 핵심 산출물은 *정리된 wiki artifact*다.
 - 즉, runtime에서 agent가 계속 기억을 읽고 보강하고 관리하는 루프는 GBrain보다 약하다.

GBrain

- 강점
 - ingest, enrich, maintain, query, citation-fix, cron, task orchestration이 연결된 구조다.
 - 사람, 회사, 미팅, 아이디어를 지속적으로 축적하고, 시간이 지나도 brain이 스스로 더 좋아지게 설계되어 있다.
 - 장기 기억을 "저장"이 아니라 "운영"하는 접근이다.
- 약점
 - 시스템이 가장 무겁다.
 - 설치, 운영, 정책 설계, API key 관리 등 진입장벽이 높다.

4.2 외부 여러 사람이 공유 가능한 knowledge base 관점

AKB

- 강점
 - 여러 agent가 동일 지식을 공유하는 backend로 적합하다.
 - 조직 내 agent, workflow, app을 연결하는 기억 인프라 계층이 될 수 있다.
- 약점
 - 사람에게 바로 보여줄 수 있는 knowledge base 경험은 약하다.
 - 검색, 문서형 탐색, 위키 링크, 퍼블리시, 읽기 흐름 측면이 보강되어야 한다.

Ilmwiki

- 강점
 - 이 영역에서는 가장 강하다.
 - static site, search, wikilink, AI export, HTML/TXT/JSON sibling, Ilms.txt, graph.jsonld까지 갖춰 공유 가능한 knowledge base 제품/품에 가깝다.
 - 내부 공유, GitHub Pages 배포, 링크 공유에 매우 적합하다.
- 약점
 - 협업 knowledge base는 강하지만, 여러 agent가 같은 live memory를 실시간으로 쓰는 backend로는 상대적으로 부적합하다.

GBrain

- 강점
 - publish 기능과 MCP 제공으로 공유 자체는 가능하다.
 - brain-first 운영을 통해 공유 전에 지식을 더 정제할 수 있다.
- 약점
 - 기본 철학이 "모든 사람이 읽는 위키"보다는 "강한 brain을 가진 agent"에 더 가깝다.
 - 그래서 knowledge base product로서는 Ilmwiki보다 직관성이 떨어진다.

4.3 기타 feature 관점

항목	AKB	Ilmwiki	GBrain
기본 아키텍처 강점	shared backend	compiled wiki	autonomous brain
검색 계층	DB/MCP 중심	wiki 탐색 + search index	multi-layer brain query
출판 / 공유	약함	매우 강함	중간
정합성 / lint	제한적	강함	강함
엔티티 확장	제한적	위키 페이지 중심	tiered enrichment 강함
스케줄링 / recurring job	약함	제한적	강함
운영 오버헤드	낮음	낮음~중간	높음

5. 프로젝트별 핵심 장단점

AKB

장점

- multi-agent shared memory라는 포지션이 분명하다.
- DB와 MCP를 중심으로 한 인프라형 설계가 자연스럽다.
- 가장 가볍게 시작할 수 있다.

단점

- 공개 기준으로는 product layer가 얇다.

- 사람에게 보여지는 knowledge base 경험이 약하다.
- memory의 품질 관리, 수명주기 관리, 정리 자동화가 부족해 보인다.

llmwiki

장점

- 사람과 AI가 함께 읽는 문서형 결과물이 뛰어나다.
- transcript를 버리지 않고 reusable knowledge로 만든다.
- 배포, 공유, 검색, 위키 탐색 경험이 강하다.

단점

- shared memory backend로는 다소 간접적이다.
- 실시간 agent memory 운영체제라기보다 knowledge compiler에 가깝다.

GBrain

장점

- 장기기억 운영 체계가 가장 완성도 높다.
- ingestion, enrichment, maintenance, scheduling이 유기적으로 연결되어 있다.
- proactive agent 운영에 매우 강하다.

단점

- 가장 무겁고 복잡하다.
- 단순한 공유 KB가 필요한 조직에는 과할 수 있다.
- 개인/특정 agent brain에 최적화된 성격이 강하다.

6. 종합 판단

세 프로젝트는 경쟁 관계이면서도, 사실은 서로 다른 층위를 가장 잘 푼 사례라고 보는 것이 맞다.

- AKB는 공유 memory infrastructure layer를 잘 정의한 접근이다.
- llmwiki는 human-readable knowledge publishing layer를 잘 정의한 접근이다.
- GBrain은 autonomous memory operations layer를 가장 강하게 구현한 접근이다.

즉, 세 접근은 아래처럼 이어진다.

1. AKB가 canonical shared memory를 저장한다.
2. 그 위에서 GBrain식 maintenance / enrichment engine이 memory를 가꾼다.
3. 최종적으로 llmwiki식 publish layer가 사람이 읽는 knowledge base를 만든다.

7. AKB 발전 방향 제안

핵심 제안은 단순하다.

AKB는 "shared memory DB"에서 멈추지 말고, "shared memory platform"으로 올라가야 한다.

7.1 제품 포지셔닝 제안

AKB의 가장 좋은 포지션은 아래와 같다.

- 현재 강점 유지: 여러 agent가 공통으로 쓰는 shared memory backend
- 중기 확장: memory quality management와 lifecycle management 추가
- 장기 확장: 사람 공유용 wiki / briefing / report layer 추가

즉, AKB는 GBrain과 IImwiki를 그대로 복제하기보다, 아래 포지션을 취하는 것이 낫다.

AKB = 조직용 *shared memory substrate* + *memory governance layer* + *publish connectors*

7.2 우선순위별 기능 제안

Phase 1. AKB의 현재 강점을 강화

1. *memory schema 표준화*
 - note, entity, source, meeting, decision, task, fact, claim 등의 타입 정의
2. *lifecycle 관리 추가*
 - draft, active, stale, archived 같은 상태값 도입
3. *citation / provenance 필드 의무화*
 - 기억의 출처와 갱신 시점을 명시
4. *collection / workspace / tenant 모델 강화*
 - 팀, 프로젝트, agent별 경계 명확화

Phase 2. GBrain식 운영 기능 흡수

1. *maintenance jobs*
 - stale memory 탐지
 - orphaned memory 탐지
 - duplicate / contradiction 탐지
2. *entity enrichment*
 - 사람, 회사, 프로젝트 페이지 자동 확장
3. *brain-first query policy*
 - 외부 검색 전에 AKB를 먼저 조회하는 정책 내장
4. *scheduled briefing / digest*
 - 최근 업데이트, 위험 이슈, 열린 질문 자동 요약

Phase 3. IImwiki식 publish layer 확장

1. *human-readable wiki export*
 - markdown / HTML / static site export
2. *AI-readable export*
 - IIm.txt, JSON, graph export
3. *공유 가능한 index / overview / timeline 페이지*
 - 프로젝트별 briefing, 변경 로그, 결정 이력 페이지 생성
4. *permission-aware publishing*
 - 내부용, 외부공유용, 고객공유용 등 뷰 분리

7.3 추천 아키텍처

AKB의 이상적인 구조는 아래와 같다.

Layer 1. Canonical Memory Layer

- PostgreSQL + vector + structured metadata
- 모든 agent의 공용 기억 원본 저장소

Layer 2. Memory Operations Layer

- dedupe, contradiction check, stale detection
- citation repair, entity enrichment, periodic maintenance

Layer 3. Retrieval / MCP Layer

- agent가 읽고 쓰는 표준 인터페이스
- multi-agent orchestration 대응

Layer 4. Publish Layer

- wiki export
- digest / report / timeline 생성
- 사람 친화적 검색 UI

이 구조로 가면 AKB는 backend의 장점을 유지하면서도, *llmwiki*와 *GBrain*의 핵심 장점을 선택적으로 흡수할 수 있다.

8. 최종 권고

최종 권고는 아래와 같다.

선택 기준

- 장기기억형 *proactive agent*가 우선이면 *GBrain*이 가장 강하다.
- 사람 중심 *shared knowledge base*가 우선이면 *llmwiki*가 가장 강하다.
- 여러 agent의 공용 기억 인프라가 우선이면 AKB가 가장 자연스럽다.

AKB에 대한 권고

AKB는 방향만 잘 잡으면 가장 전략적 가치가 클 수 있다. 이유는 아래와 같다.

1. 조직에서는 agent가 하나가 아니라 여러 개가 된다.
2. 따라서 공용 memory substrate의 중요성이 커진다.
3. 다만 backend만으로는 차별화가 약하다.
4. 그래서 *memory governance*, *maintenance*, *publishability*까지 올라가야 한다.

즉, AKB의 정답은 아래 한 문장으로 요약된다.

AKB는 "shared vector memory"가 아니라, "조직의 공용 agent memory operating platform"이 되어야 한다.

9. 참고 자료

1. Andrej Karpathy, *llm-wiki* pattern gist
 - <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>
2. Pratiyush, *llmwiki* README
 - <https://github.com/Pratiyush/llm-wiki>
3. Garry Tan, *GBrain* README
 - <https://github.com/garrytan/gbrain>
4. LobeHub MCP listing, AKB — Agent Knowledge Base
 - <https://lobehub.com/mcp/foose212-akb-server>

10. 한 줄 결론

- *GBrain*은 가장 강한 장기기억 엔진이다.
- *llmwiki*는 가장 좋은 공유형 knowledge base 결과물이다.
- AKB는 가장 좋은 shared memory substrate다.
- AKB의 발전 방향은, 여기에 *maintenance*와 *publish layer*를 붙여 조직용 memory platform으로 진화하는 것이다.